

Documentación del Formulario de Registro de Productos

1. Formulario Creado

Estructura del Formulario Reactivo

Se implementó un formulario reactivo utilizando `FormGroup` y `FormControl` de Angular para gestionar el registro de productos. El formulario contiene los siguientes campos:

|`nombre` | Texto | Nombre del producto (3-50 caracteres)

|`descripcion` | Texto | Descripción detallada (10-200 caracteres)

|`precio` | Número | Precio en quetzales (0.01-100000)

|`categoria` | Texto | Categoría del producto (mínimo 3 caracteres)

|`stock` | Número | Cantidad en inventario (0-10000)

Inicialización del Formulario

```
initForm(): void {  
  this.productoForm = this.fb.group({  
    nombre: ['', [Validators.required, Validators.minLength(3), Validators.maxLength(50)]],  
    descripcion: ['', [Validators.required, Validators.minLength(10),  
Validators.maxLength(200)]],  
    precio: [0, [Validators.required, Validators.min(0.01), Validators.max(100000)]],  
    categoria: ['', [Validators.required, Validators.minLength(3)]],  
    stock: [0, [Validators.required, Validators.min(0), Validators.max(10000)]]  
  });  
}
```

2. Validaciones Definidas

Validadores Utilizados

Se implementaron validaciones síncronas utilizando los validadores integrados de Angular:

Campo: nombre

Validators.required: Campo obligatorio

Validators.minLength(3): Mínimo 3 caracteres

Validators.maxLength(50): Máximo 50 caracteres

Campo: descripcion

Validators.required: Campo obligatorio

Validators.minLength(10): Mínimo 10 caracteres

Validators.maxLength(200): Máximo 200 caracteres

Campo: precio

Validators.required: Campo obligatorio

Validators.min(0.01): Valor mínimo 0.01 (no permite 0 o negativos)

Validators.max(100000): Valor máximo 100000

Campo: categoria

Validators.required: Campo obligatorio

Validators.minLength(3): Mínimo 3 caracteres

Campo: stock

Validators.required Campo obligatorio

Validators.min(0): No permite valores negativos

Validators.max(10000): Máximo 10000 unidades

Gestión de Estados de Control

Se utilizan los estados del formulario para controlar la visualización de errores:

touched: El usuario ha interactuado con el campo

invalid: El campo no cumple con las validaciones

isSubmitted: El usuario intentó enviar el formulario

// Solo muestra errores si el campo fue tocado O si se intentó enviar

```
[class.is-invalid]="f['nombre'].invalid && (f['nombre'].touched || isSubmitted)"
```

3. Uso de *ngIf y *ngFor

Directiva *ngIf

Se utiliza para el renderizado condicional de elementos:

Visualización de Mensajes de Error

```
<div *ngIf="f['nombre'].invalid && (f['nombre'].touched || isSubmitted)" class="error-  
messages">  
  <span *ngFor="let error of getErrors('nombre')" class="error-text">• {{ error }}</span>  
</div>
```

Componente de Alerta de Éxito

```
<app-alerta-exito  
  *ngIf="mostrarAlerta"  
  titulo="Producto registrado"  
  mensaje="El producto se ha agregado correctamente al sistema">  
</app-alerta-exito>
```

Estado Vacío en la Lista de Productos

```
<div *ngIf="filteredProductos.length === 0" class="empty-state">
  <p>No hay productos registrados</p>
</div>
```

Directiva *ngFor

Se utiliza para iterar sobre colecciones:

Iteración de Mensajes de Error

```
<span *ngFor="let error of getErrors('nombre')" class="error-text">
  • {{ error }}
</span>
```

Renderizado de la Tabla de Productos

```
<tr *ngFor="let producto of filteredProductos">
  <td class="font-medium">{{ producto.nombre }}</td>
  <td><span class="badge">{{ producto.categoria }}</span></td>
  <td class="text-muted">{{ producto.descripcion }}</td>
  <td class="font-medium text-success">Q{{ producto.precio | number:"1.2-2" }}</td>
  <td>
    <div class="stock-bar-container">
      <span>{{ producto.stock }}</span>
      <div class="stock-bar-bg">
        <div class="stock-bar-fill"
          [ngClass]="getStockPercentage(producto.stock) >= 80 ? 'high' :
            (getStockPercentage(producto.stock) >= 50 ? 'medium' : 'low')"
          [style.width.%]="getStockPercentage(producto.stock)">
        </div>
      </div>
    </div>
  </td>
</tr>
```

4. Envío y Verificación de Datos

Flujo de Envío de Datos

1. Validación en el Frontend

```
onSubmit(): void {  
  this.isSubmitted = true;  
  // Validación del formulario  
  if (this.productoForm.invalid) {  
    this.productoForm.markAllAsTouched();  
    return;  
  }  
  const nuevoProducto: Producto = this.productoForm.value;  
  // Envío al backend  
  this.productoService.addProducto(nuevoProducto).subscribe({  
    next: (response) => {  
      console.log("Producto registrado exitosamente:", response);  
      this.mostrarAlerta = true;  
      this.productoForm.reset();  
      this.isSubmitted = false;  
      setTimeout(() => {  
        this.router.navigate(["/productos"]);  
      }, 1500);  
    },  
    error: (error) => {  
      console.error("Error al registrar el producto:", error);  
    }  
  });  
}
```

2. Servicio de Comunicación (producto.service.ts)

```
addProducto(producto: Producto): Observable<Producto> {  
  return this.http.post<Producto>(this.apiUrl, producto);  
}
```

El servicio utiliza `HttpClient` para realizar una petición POST a la API del backend.

3. Controlador del Backend (producto.controller.ts)

```
export const agregarProducto = async (req: Request, res: Response) => {  
  const { nombre, descripcion, precio, categoria, stock } = req.body;  
  
  try {  
    const result = await pool.query(  
      "INSERT INTO productos (nombre, descripcion, precio, categoria, stock) VALUES ($1, $2, $3, $4, $5) RETURNING *",  
      [nombre, descripcion, precio, categoria, stock]  
    );  
  
    console.log("Producto agregado al backend:", result.rows[0]);  
    res.status(201).json(result.rows[0]);  
  } catch (error) {  
    console.error("Error al agregar producto:", error);  
    res.status(500).json({ message: "Error al agregar producto" });  
  }  
};
```

Verificación de Datos

En el Frontend:

1. Validación de formulario: Los validadores de Angular verifican que los datos cumplan con las reglas definidas antes del envío
2. Console.log: Se imprime el objeto producto antes de enviarlo para verificar su estructura
3. Respuesta del servidor: Se verifica la respuesta exitosa en el bloque `next` del observable

En el Backend:

1. Extracción de datos: Se extraen los campos del `req.body`
2. Query parametrizada: Se utilizan parámetros (`\$1`, `\$2`, `\$3`, `\$4`, `\$5`) para prevenir inyección SQL
3. RETURNING: La consulta retorna el registro insertado para confirmar la operación
4. Manejo de errores**: Se captura cualquier error de base de datos y se retorna un mensaje apropiado
5. Código de estado HTTP: Se retorna `201 Created` para indicar éxito en la creación

Ejemplo de Datos Enviados

```
{
  "nombre": "Teclado Mecánico",
  "descripcion": "Teclado con switches azules y retroiluminación",
  "precio": 450.00,
  "categoria": "Periféricos",
  "stock": 15
}
```

Respuesta del Backend

```
{
  "id": 1,
  "nombre": "Teclado Mecánico",
  "descripcion": "Teclado con switches azules y retroiluminación",
  "precio": "450.00",
  "categoria": "Periféricos",
  "stock": 15,
  "fecha_creacion": "2026-09-05T03:12:54.312Z",
  "fecha_actualizacion": "2026-09-05T03:12:54.312Z"
}
```